

Rapport de soutenance J3D

Echoes Of Lights

-

Les Debuggers

Sommaire :

1. Review du TSD	2 - 4
2. Rapports personnels	4 - 21
2.1. Antoine LAPP	5 - 7
2.2. Guillaume KLEIN	7 - 13
2.3. Antoine MUH	14 - 15
2.4. Gustave SCHWIEN	16 - 19
2.5. Valentin GOUSSOT	20 - 21
3. Récit de mise en œuvre du projet	21 - 22
3.1. Challenges	21 - 22
3.2. Réussites / Succès	22 - 22
4. Annexes	23 - 25

1. Review du TSD :

Dans cette partie du rapport, nous présentons une analyse complète de l'avancement du projet en comparant les objectifs que nous nous étions fixés avec l'état réel de chaque composant du jeu. Cette évaluation couvre l'ensemble des domaines majeurs du développement, tels que le pixel art, l'animation, le gameplay, l'intelligence artificielle, l'affichage ou encore les fonctionnalités multijoueurs. Elle permet de mettre en évidence les points où nous sommes en avance, ceux où nous sommes en retard, ainsi que les raisons qui expliquent ces écarts. L'objectif est de fournir une vision claire et structurée de la progression du projet, afin de mieux orienter les priorités pour la suite du développement.

On peut retrouver le tableau initial du TSD en annexe 1.1

Pixel Art

Prévision : 50 % Avancement réel : 40 %

Comparaison: Nous avons produit les tuiles de terrain ainsi que les spritesheets des joueurs. Cependant, une grande partie de notre temps a été consacrée au développement du gameplay et des énigmes, qui se sont révélés plus complexes que prévu. Cela a entraîné un léger retard sur la partie artistique, même si les éléments essentiels sont déjà en place.

Musique

Prévision : 50 % Avancement réel : 50 %

Comparaison: La musique principale du jeu a été composée par Valentin, et nous disposons également d'une petite playlist générée par IA. L'avancement correspond parfaitement à nos prévisions et cette partie du projet ne devrait poser aucune difficulté pour la suite.

Animation

Prévision : 50 % Avancement réel : 30 %

Comparaison: Les spritesheets des joueurs sont complètes pour l'ensemble de leurs mouvements. Une première version de l'animation d'attaque de mêlée est également disponible, même si elle sera améliorée ultérieurement. Nous aurions souhaité avancer davantage, notamment sur les animations des ennemis, ce qui explique le retard constaté.

Sound Design

Prévision : 50 % Avancement réel : 0 %

Comparaison: Comme pour le pixel art, le développement du gameplay a nécessité plus de temps que prévu. Nous avons donc mis le sound design de côté pour le moment. Cette partie sera plus simple à intégrer plus tard dans le projet, et même si nous sommes en retard, la charge de travail restante ne devrait pas poser de problème d'ici la soutenance finale.

Multijoueur

Prévision : 100 % Avancement réel : 100 %

Comparaison: Le multijoueur était déjà bien avancé lors de la précédente soutenance. Durant cette période, nous avons finalisé le partage et la gestion des actions entre les joueurs. La partie réseau est désormais terminée. Seuls quelques ajustements mineurs pourront être nécessaires par la suite, mais aucun changement majeur n'est prévu.

Gameplay

Prévision : 90–100 % Avancement réel : 60 %

Comparaison: Le gameplay s'est révélé beaucoup plus complexe à développer que prévu, ce qui explique notre retard. Nous souhaitons intégrer deux compétences ainsi que l'attaque de mêlée, mais seule cette dernière est actuellement fonctionnelle.

Concernant les énigmes, nous devons en avoir deux opérationnelles : les leviers et le Jeu de la Vie (voir rapport 1 – Gustave). Pour l'instant, seule l'énigme des leviers est terminée.

Nous avons également un ennemi capable d'attaquer, mais il n'est pas encore possible de lui infliger des dégâts ou de le vaincre.

Map

Prévision : 50 % Avancement réel : 80 %

Comparaison: L'intégralité des murs de la carte est en place, chaque salle est correctement délimitée et l'ensemble est stocké dans une image en pixel art. Il ne manque plus que des éléments de décor pour rendre l'environnement plus agréable et moins géométrique.

Déroulement du jeu

Prévision : 100 % Avancement réel : 100 %

Comparaison: Le déroulement complet du jeu est défini sur le papier. C'est notre fil conducteur depuis le début du projet, et nous sommes donc totalement à jour sur cette partie. Le principal défi reste désormais de traduire ces idées en code.

Affichage du jeu

Prévision : 100 % Avancement réel : 100 %

Comparaison: Le moteur isométrique est entièrement fonctionnel. Seuls quelques ajustements mineurs seront réalisés au fil du développement, mais la structure générale est finalisée. Nous pouvons ajouter ou modifier des tiles très facilement grâce à l'architecture modulaire mise en place.

Développement des joueurs

Prévision : 90 % Avancement réel : 70 %

Comparaison: Nous avons prévu d'intégrer les deux compétences, l'attaque de mêlée, les animations de déplacement et les interactions avec l'environnement. Si les animations et les interactions sont terminées, les compétences n'ont pas encore été implémentées, ce qui explique notre léger retard.

Affichage des menus

Prévision : 100 % Avancement réel : 100 %

Le menu principal était déjà finalisé lors de la dernière soutenance. Nous avons ajouté le menu pause, et aucun ajout majeur n'est prévu. Quelques ajustements mineurs pourront être faits en fonction des futures fonctionnalités, notamment l'ajout d'options dans le menu pause.

IA

Prévision : 70% Avancement réel : 80%

Comparaison : Le pathfinding avec l'algorithme A* est fonctionnel : l'ennemi trouve le meilleur chemin pour se rendre à la position du joueur cible tout en évitant les obstacles présents sur la map, et se déplace en autonomie selon le chemin retenu. L'ennemi peut se déplacer et attaquer dans 8 directions différentes. Son IA adapte son comportement en fonction de sa distance par rapport au joueur : il se met à attaquer le joueur lorsque celui-ci se trouve à sa portée, en prenant en compte la position du joueur par rapport à la sienne pour attaquer dans la bonne direction. Sinon, il le chasse avec A*.

Le plus gros du travail a été réalisé. Il reste maintenant à optimiser le code : nous devons corriger des bugs d'affichage (ex : ennemi passant légèrement "sur" un obstacle), peut-être revoir la forme des hitbox pour les adapter à l'affichage isométrique, adapter le pathfinding pour d'autres ennemis comme le Flinger qui enverra des projectiles à distance (il aura une portée d'attaque bien plus grande que le Slasher).

2. Rapports personnels

2.1. Antoine LAPP :

Durant cette période, je me suis encore une fois occupé principalement de l'aspect réseau du jeu mais cette fois-ci, mes tâches n'étaient pas uniquement liées au multijoueur, et j'ai dû adapter le code de certains de mes camarades pour le faire fonctionner avec mon code réseau et ainsi partager les données des différents éléments du jeu et les synchroniser entre les 2 joueurs.

1. Affichage en plein écran

Ma première tâche a été de permettre l'affichage en plein écran du jeu. Bien que le code du jeu s'adaptait déjà en partie selon les dimensions de l'écran, j'ai dû transformer les coordonnées statiques de tous les éléments du menu en coordonnées relatives pour qu'elles évoluent en fonction du format de l'écran quand on met le jeu en plein écran : les dimensions de chaque éléments du menu sont définis par des fraction de la largeur et de la longueur de la fenêtre du jeu (et donc de l'écran quand on est en mode plein écran). Après réflexion, nous avons décidé d'imposer aux utilisateurs d'être en mode pleine écran tout le long du jeu afin d'éviter de devoir gérer des modifications de la taille de l'écran au cours de l'exécution du programme. Cependant, pour faciliter le développement, qui nécessite parfois d'avoir plusieurs instances du jeu lancées, ou qui nécessite de pouvoir voir à la fois le jeu et le code source, j'ai laissé un mode fenêtré (non redimensionnable). On peut donc décider de mettre le jeu en plein écran ou non via un booléen dans le code, selon si le jeu est en développement ou s'il est dédié à la production.

2. Menu pause et gestion en cas de départ d'un joueur

Ensuite, je me suis chargé de développer un menu pause accessible par le joueur via la touche échap. Une capture d'écran de ce menu pause est disponible dans l'annexe (2.1.1.). Ce menu permet au joueur de fermer le jeu ou de retourner au menu principal. Afin d'éviter de dupliquer du code ou de faire des imports cycliques, j'ai déplacé les différents éléments de menu dans un fichier à part `menu_components.py`. Développer cette partie m'a aussi demandé de gérer la situation où un des 2 joueurs quitte la partie. Étant donné que l'on souhaite que notre jeu se joue impérativement à 2, il faut donc forcer l'autre joueur à quitter également et à revenir au menu principal. Pour cela j'ai donc d'abord dû gérer le fait de fermer proprement le datachannel dans lequel transit les données entre les joueurs, ainsi que le fait de prévenir l'autre joueur du départ de son partenaire afin de le forcer à revenir au menu. Lorsqu'un joueur quitte, il envoie dans le datachannel un message contenant `"player_quitting"` ensuite il ferme le datachannel de son côté. L'autre joueur, lorsqu'il reçoit le message précédemment cité, il ferme lui aussi le datachannel de son côté et comme la boucle principale du jeu arrête de tourner, il est automatiquement renvoyé au menu. C'est durant le développement de cette fonctionnalité que je me suis rendu compte qu'un des choix que j'avais fait pour le multijoueur n'était pas le plus optimal : avant, le joueur dit client envoyait toutes ses données dès qu'il recevait celle du joueur host. Mais donc s'il y avait un problème et que le joueur host ne lui envoyait plus rien, lui aussi n'envoyait plus rien. J'ai donc fait en sorte que dans tous les cas, les deux joueurs envoient leurs données à une fréquence régulière adaptable (actuellement 60 FPS mais on réduira s'il est nécessaire d'optimiser) indépendamment de l'autre joueur. J'ai aussi dû gérer le fait de réinitialiser tous les différents éléments du jeu lorsque les joueurs reviennent au menu principal afin de pouvoir relancer une nouvelle partie.

3. HUD et système de points de vie

Après cela, je me suis occupé d'implémenter le système de points de vie des joueurs et de l'affichage de ceux-ci dans le HUD du jeu. Au départ j'étais partie sur un affichage sous forme de cœur comme dans le jeu Minecraft, mais après réflexion avec mes camarades, on a préféré partir sur une barre plus ou moins remplie indiquant le nombre de points de vie, celle-ci est visible dans l'annexe (2.1.2.). J'ai encore une fois fait en sorte de pouvoir activer ou désactiver l'affichage du HUD afin de faciliter l'expérience de développement. Afin d'implémenter ce système de points de vie, j'ai d'abord créé une superclass `Entity()` qui est classe mère des classe `Player` et `Enemy` afin que celles-ci puissent partager des attributs tels que la position et les points de vie. J'ai donc également développé la méthode `take_damage()` de cette superclass afin de retirer des points de vie. L'affichage se met automatiquement à jour en calculant le ratio points de vie actuel sur points de vie maximum afin de définir la longueur de la barre.

4. Système de portes pour les énigmes

Ensuite, je me suis chargé d'implémenter le système de portes pour les énigmes, j'ai dû réfléchir à comment les représenter dans la matrice de la map et comment faire en sorte de les placer facilement dedans. À l'image des leviers de Guillaume, chaque porte est représentée dans l'image de la map par un pixel dont 1 octet (celui représentant le bleu dans la couleur du pixel) sert à définir dans quel groupe elle appartient afin de pouvoir en ouvrir plusieurs en même temps. Un autre octet (celui représentant la couleur verte) peut prendre 8 valeurs : 2 pour chacune des 4 orientations, une pour chaque orientation d'ouverture (les valeurs associées sont visibles dans le tableau 2.1.3. de l'annexe). Les portes sont donc représentées dans le code par une classe `Door` possédant des coordonnées, un attribut `state` qui est un booléen décrivant si la porte est ouverte ou fermée, et un attribut `orientation` qui est une liste qui contient les deux numéro de taille à implanter dans la map. J'ai donc fait une méthode `open_close()` qui inverse l'état de la porte et appelle une action qui modifie la matrice de la map afin d'ouvrir ou fermer la porte. Dans cette matrice, les portes sont représentées par 8 numéros : 2 par orientation, une pour quand la porte est ouverte et une pour quand elle est fermée. J'ai également dû modifier le système de collision pour permettre aux joueurs de passer dans les portes ouvertes, Guillaume l'a ensuite amélioré. Pour implémenter ce système de porte, j'ai aussi dû désigner des tuiles de tests pour celle-ci. Au final on a donc bien des portes rassemblées dans des groupes afin de les synchroniser, elles sont disponibles dans toutes les orientations et leur état est synchronisé entre les 2 joueurs. Leur représentation ouverte et fermée est visible dans les captures 2.1.4. de l'annexe.

5. Implémentation des spritesheets et aide pour le multijoueur des ennemis

Je me suis également occupé d'implémenter la spritesheet désigné par Antoine Muh afin d'avoir des animations fluides et agréables selon les mouvements des joueurs. J'ai donc créé une classe `Spritesheet` afin de récupérer les animations et les frames pour par la suite les appliquer en fonction des déplacements. Il a également fallu que les états du joueur (direction et à l'arrêt ou en train de marcher) soient envoyés sur le réseau afin que les animations soient aussi jouées sur l'écran de l'autre joueur.

Enfin, je me suis aussi occupé d'aider Gustave à implémenter la partie réseau pour ses ennemis, afin de partager toutes leurs informations et de répliquer leurs mouvements et attaques chez les 2 joueurs. Après réflexion, on a décidé que pour l'instant, seul le joueur host se chargerait de décider des actions des ennemis par soucis de simplicité et de rapidité d'implémentation, mais, si c'est nécessaire pour plus d'optimisation par la suite, on fera en sorte de répartir les décisions entre les 2 joueurs. J'ai aussi encore une fois rajouté un paramètre de développement pour l'affichage des hitboxs qu'a développé Gustave.

6. Site web

J'ai aussi rajouté sur le site web le bouton qui permet de télécharger ce rapport et j'ai également ajouté les nouvelles ressources utilisées pendant cette période dans la page dédiée aux sources.

Conclusion et tâches futures

Pour finir, sur cette période, j'ai encore appris beaucoup de chose, mais contrairement à la période précédente, cette fois-ci j'ai eu à régler beaucoup de conflits et à vraiment travailler avec les autres : il a fallu comprendre leur code afin de l'appliquer au réseau mais aussi discuter afin de prendre différentes décisions pour le développement. Par exemple, nous avons dû beaucoup discuter pour choisir comment nous allions partager les différents états des énigmes. Grâce à ces discussions, nous avons pu choisir d'implémentations d'actions qu'on partage sur le réseau et qu'on exécute chez les 2 joueurs quasiment en même temps. C'est selon nous la façon la plus facilement répliquable.

En ce qui concerne les actions que je vais mener dans la suite de ce projet, je vais sûrement continuer à aider mes camarades à appliquer leur code au multijoueur et s'il est nécessaire, chercher à optimiser le code réseau afin de réduire une potentielle latence, notamment en réduisant le nombre d'envois par seconde ou en répartissant les tâches entre les 2 joueurs. Je vais également aider Guillaume et Gustave à mettre en place les différentes énigmes et développer les nouvelles fonctionnalités pour celle-ci comme par exemple en créant de nouveaux objets interactifs. Je vais également commencer à me renseigner sur les moyens de transformer notre code en un unique exécutable à télécharger afin d'éviter aux futurs utilisateurs de devoir lancer le code dans un terminal et d'installer les dépendances externes avec la commande pip.

Ressources

- <https://www.pygame.org/docs/>
- https://www.youtube.com/watch?v=M6e3_8LHc7A
- <https://www.piskelapp.com/>

2.2. Guillaume KLEIN :

1. Génération de la carte à partir d'un pixel art et GameRegistry

Afin de simplifier la création et la maintenance des niveaux, la carte du jeu est définie à partir d'une image pixel art. Lors de l'initialisation de la partie le jeu charge l'image de la map qui est défini la taille de la matrice du jeu au passage, les largeurs et hauteurs correspondent tandis que la hauteur est défini à 3 par défaut dans le jeu (les murs font 2 de hauteur sans compter le sol). Le moteur isométrique parcourt chacun des pixels de l'image afin de créer chacune des cases de la matrice. Chacun des pixels de cette image nous sert à stocker les informations tels que, les emplacements des portes, des leviers, le sens d'affichage, le groupe de l'objet. En fonction des différentes valeurs dans les champs Rouge Vert Bleu, on peut déduire les caractéristiques de chacun de nos objets. Celles-ci sont directement ajoutées dans notre matrice. Cette matrice contient alors pour l'instant la position des sols, des murs, des portes, des leviers.

Afin de stocker toutes les informations nécessaires à la progression des joueurs, j'ai créé `GameRegistry` qui est une classe dont l'instance permet de stocker l'État du jeu et de la progression des joueurs, les ennemis, les leviers, les portes. Ce `gameRegistry` est directement rempli à l'initialisation de la carte par le moteur isométrique pour ce qui est des leviers et des portes (ainsi que pour les autres énigmes par la suite).

L'objectif à terme est de pouvoir produire des sauvegardes de parties pour les joueurs à partir de cette instance du `GameRegistry` en JSON, dans le but de pouvoir recharger l'État du jeu dans lequel ils étaient avant de quitter leur partie. Nous pourrions proposer dans le menu de charger une sauvegarde qui serait stockée dans un fichier avec un format particulier, idéalement illisible et non modifiable par les joueurs pour éviter toute triche.

Utilisation des couleurs RGB

Comme introduit avant, chaque composante de la couleur RGB peut être utilisée pour transmettre des informations supplémentaires, dans mon implémentation je me sers des deux premiers champs, Rouge et Vert pour caractériser l'objet, son type, les différentes variations possibles, le dernier champs qui est le Bleu sert de nombre du groupe, l'identifiant de la salle. Par exemple, deux leviers ayant la même valeur Blue appartiennent à la même énigme, les portes possédant le même identifiant de groupe peuvent être liées à ces leviers.

De manière totalement arbitraire nous avons choisi le nombre 46 pour la valeur du blue sur les pixel de la première énigme, ainsi tous les éléments ayant 46 en valeur pour le bleu dans le pixel seront en liens avec la première énigme (par la suite on normalisera les nombres de 1 au nombre de salles dans la carte pour rester clair)

Cela permet de différencier facilement les différentes salles, d'éviter de coder manuellement les relations entre objets et de modifier les niveaux simplement en modifiant l'image pixel art. Ainsi, le level design devient beaucoup plus simple, modifier la carte revient simplement à éditer une image, sans modifier le code. En changeant simplement une valeur du pixel RGB on peut modifier les relations avec l'environnement, supprimer et ajouter de l'interaction / difficulté.

Le jeu contiendra de nombreuses salles. Il est donc nécessaire de pouvoir distinguer facilement les objets appartenant à une même énigme, sans devoir coder manuellement chaque relation. Aussi cela nous permettra d'obtenir une mini carte du jeu à présenter sur le site Web pour donner un avant goût aux possibles clients.

2. Logique générale des actions et interactions

Pour structurer les interactions du jeu et garantir leur cohérence dans un contexte multijoueur coopératif, j'ai développé un système d'actions centralisé basé sur une architecture orientée objet. Toutes les interactions du jeu qu'elles proviennent des joueurs, des ennemis ou de l'environnement (leviers, portes, mécanismes) reposent sur des actions qui appliqueront les conséquences à l'environnement du jeu. Ces actions sont toutes issues d'une classe mère `Action`, celle-ci pose la base de toutes les interactions possibles avec l'environnement.

Chaque compétence héritant de cette classe représente une action possible dans le jeu. Elle possède notamment une méthode principale : `execute()`

Cette méthode applique les conséquences de l'action sur l'environnement ou les entités du jeu. On a pour l'instant différentes actions disponibles telles que: `EditMapAction`, `MeleeAction`, `LeverAction` et `InteractAction`. Pour les interactions dans l'environnement hors joueurs et ennemis, ces Actions sont gérées et exécutées directement en local tandis que pour les joueurs, on dispose de Skills qui sont stockés dans une liste au sein de la classe `Joueur`, ce sont des instances de classes, héritières de la grande classe mère `Skill` qui permet la gestion du cooldown, de la disponibilité de la compétence et de la création de l'Action qui va appliquer les changements à l'environnement.

Lors de la création d'une action depuis le `Skill` d'un joueur, celle-ci est envoyée dans la file d'exécution du `game_context` qui va stocker toutes les actions et vider la file afin de les exécuter. C'est au moment de l'exécution, par le biais de la méthode `send_to_network()` dans notre classe `Action` que l'action est envoyée à l'autre joueur pour synchroniser l'État du monde. Le fonctionnement global du système est le suivant :

1. Un joueur (ou une entité) utilise un `Skill`.
2. Le `Skill` crée une instance d'une `Action` contenant les paramètres nécessaires.
3. Cette Action est ajoutée dans une file d'exécution appartenant au contexte de jeu.

4. Le contexte vide régulièrement cette file et exécute les actions dans l'ordre.
5. Les actions sont alors envoyés à l'autre joueur

voir schéma en annexe 2.2.2

Ce système permet plusieurs avantages importants : Centralisation de la logique de jeu, Ordonnancement clair des événements, une plus grande facilité à synchroniser en multijoueur, et l'extensibilité pour de nouvelles interactions. On peut résumer cette architecture ainsi : Le joueur essaie d'utiliser un de ses Skills, celui-ci crée une action si la compétence est disponible et la renvoie. Enfin cette action est récupérée depuis la boucle principale du jeu qui l'ajoute à la file du contexte pour être exécutés. A la sortie de la file lors de la fin de la boucle, l'action est exécutée et partagée au joueur en réseau.

3. Synchronisation des actions en multijoueur

Ce système d'action a donc été conçu pour résoudre un problème majeur du multijoueur : la synchronisation des interactions entre les deux joueurs. Le fait d'englober les différentes conséquences d'une interaction d'un joueur dans une instance de classe générée à partir de paramètres permet de simplifier le transfert des informations en ligne. Lorsqu'un joueur déclenche une action, celle-ci doit être reproduite exactement sur la machine du second joueur, cependant, il n'est pas possible d'envoyer directement une instance Python d'une action à travers le réseau.

La solution mise en place consiste donc à transmettre le nom de l'action ainsi que les paramètres nécessaires à son exécution. A la réception, l'autre client reçoit ces informations : Il recrée localement la même action à partir du nom et des paramètres reçus. Cette action est ajoutée dans sa propre file d'actions. Elle est exécutée dans le `game_context` comme si elle avait été créée localement. L'utilisation d'un paramètre `Host` à la création des actions permet d'éviter une boucle infini de renvoi d'action entre les deux joueurs. Lorsqu'une action est créée suite à la réception sur le canal du multijoueur, la classe possède un paramètre `Host` qui est mis à `False`, cela permet d'indiquer à la méthode `send_to_network()` de l'action que celle-ci ne doit pas être renvoyée.

Ce mécanisme garantit que les deux joueurs exécutent les mêmes actions, dans le même ordre, grâce aux mêmes paramètres. Cela permet le maintien d'une synchronisation des deux joueurs.

4. Implémentation de la classe Player

Afin de gérer les entités contrôlées par les joueurs, j'ai implémenté une classe `Player` héritant d'une classe plus générique appelée `Entity`. Cette classe de base regroupe les caractéristiques communes à toutes les entités du jeu, notamment les joueurs et les ennemis. Elle stocke des informations fondamentales telles que la position de l'entité, ses points de vie, sa vitesse de déplacement et sa position de réapparition. Elle contient également des méthodes utilitaires comme la gestion des dégâts ou le système de réapparition. La classe `Player` étend cette structure afin d'ajouter des comportements spécifiques aux joueurs.

La classe `Player` stocke plusieurs informations nécessaires au fonctionnement du personnage : l'avatar du joueur, utilisé pour l'affichage, un indicateur `host` permettant de savoir si le joueur est l'hôte de la partie, la direction actuelle du joueur, utilisée notamment pour certaines compétences, une liste de compétences (Skills) que le joueur peut utiliser, une hitbox, permettant de gérer les collisions. La direction du joueur est représentée sous forme d'un couple (dx, dy) correspondant à la direction du déplacement. Cette information est importante pour déterminer l'orientation de certaines actions, comme les attaques ou les interactions avec l'environnement. La classe stocke également une hitbox rectangulaire, positionnée avec un offset afin de correspondre correctement à la zone physique du personnage dans le monde du jeu.

Utilisation des compétences

Le joueur possède une liste appelée `skills`, contenant les différentes compétences qu'il peut utiliser. Une méthode appelée `try_use()` permet d'activer une compétence en fonction de son index dans la liste. Cette méthode appelle ensuite la méthode `try_use()` de la compétence correspondante. Ce système permet de : rendre les compétences modulaires afin d'en ajouter ou supprimer facilement et de connecter les compétences au système d'action du jeu. Chaque compétence gère également son propre temps de recharge (cooldown). La méthode `update()` de la classe `Player` est appelée à chaque frame et permet de mettre à jour les cooldowns de toutes les compétences du joueur.

Implémentation de l'attaque à l'épée

La compétence d'attaque au corps à corps est implémentée dans une classe appelée `SwordAttack`, qui hérite directement de la classe `Skill`. Cette compétence possède un temps de recharge spécifique ainsi qu'une portée limitée, correspondant à une attaque à courte distance. Lorsque le joueur utilise cette compétence, la méthode `create_action()` génère une instance de `MeleeAction`, qui sera ensuite exécutée par le système d'action du jeu. Cette action contient les informations nécessaires à l'exécution de l'attaque, notamment : l'entité qui réalise l'attaque (le joueur) et la portée de l'attaque, ainsi que la direction dans lequel celle-ci est exécutée. Une fois créée, l'action est ajoutée à la file d'exécution du `game_context`, comme toutes les autres actions du jeu.

Gestion des déplacements

La classe `Player` intègre également la logique de déplacement du personnage. La méthode `detect_movement()` est responsable de l'interprétation des commandes du joueur. Elle prend en paramètre : les entrées clavier et la matrice de la carte. Dans ce projet, les déplacements sont contrôlés via les touches ZQSD, correspondant aux directions haut, gauche, bas et droite. À partir de ces entrées, la méthode calcule un vecteur de déplacement (`dx`, `dy`) qui sera appliqué à la position du joueur si le mouvement est autorisé.

Afin d'obtenir une détection de collision plus précise, le système ne se base pas uniquement sur un point central du joueur. Une méthode appelée `deduce_feet_from_iso_coords()` permet de calculer la position des deux pieds du personnage à partir de ses coordonnées isométriques. Ces deux points représentent le pied gauche et le pied droit. Lorsqu'un déplacement est envisagé, la position future des deux pieds est calculée. Le mouvement n'est autorisé que si les deux positions sont valides, ce qui évite qu'un joueur puisse partiellement traverser un obstacle par exemple un mur ou une porte. Pour vérifier la possibilité du déplacement, la méthode `foot_can_move()` permet de déterminer si un pied peut se déplacer vers une position donnée. Elle convertit d'abord les coordonnées isométriques en coordonnées de la grille grâce à la fonction `iso_to_cart_tile()`. Cette conversion renvoie les coordonnées du joueur dans la matrice de la carte. La méthode `is_walkable()` est ensuite utilisée pour vérifier si la tuile correspondante est traversable.

Une tuile est considérée comme traversable dans plusieurs cas : lorsqu'il s'agit d'un sol, lorsqu'il s'agit d'une porte partiellement ouverte. Au 6. de ma partie, j'explique de quelle manière je détecte la collision avec un objet qui ne remplit pas sa tuile.

Mise à jour de la position du joueur

Si les deux pieds peuvent se déplacer vers leur position future, la position du joueur est mise à jour en appliquant le vecteur de déplacement multiplié par la vitesse. Après chaque déplacement, la position de la hitbox est également recalculée afin de rester alignée avec la position du joueur. La classe met également à

jour la direction du joueur et l'état de mouvement, utilisé pour savoir si le joueur est en train de se déplacer ou non. Ces informations sont utilisées par d'autres systèmes du jeu, notamment les compétences et l'affichage.

4. Implémentation technique des leviers interactifs

Pour implémenter l'énigme des leviers, j'ai conçu une classe dédiée appelée `Lever`. Cette classe représente un levier interactif présent dans l'environnement du jeu et contient toutes les informations nécessaires à son fonctionnement. Chaque levier stocke plusieurs informations essentielles : un identifiant unique permettant de différencier chaque levier; la position dans la carte, l'identifiant du groupe auquel il appartient, son état actuel (activé ou désactivé), un état de verrouillage empêchant toute interaction, une liste de leviers liés permettant de créer des interactions entre eux.

L'identifiant de groupe permet notamment d'associer plusieurs leviers à une même énigme ou à une même salle.

Afin de créer des énigmes complexes, les leviers peuvent être liés entre eux. Lorsqu'un levier est utilisé, il peut modifier l'état d'autres leviers. Pour cela, chaque levier possède une liste appelée `links`, contenant les autres leviers auxquels il est connecté. Deux méthodes permettent de gérer ces relations : `add_link()` qui ajoute un lien avec un autre levier et `remove_link()` qui supprime un lien existant. Ce système permet de construire dynamiquement des réseaux de leviers pouvant produire des effets en chaîne.

Interaction avec le levier

L'interaction principale avec un levier se fait via la méthode `toggle()`, qui inverse son état. Lorsqu'un levier est activé par un joueur : son état est inversé → la méthode `on_toggle()` est appelée → l'état du levier est mis à jour dans la matrice du jeu. La méthode `update_matrix()` permet de modifier la matrice utilisée par le moteur isométrique afin de changer l'apparence du levier à l'écran. Selon son état : le levier actif correspond à une tuile spécifique et le levier inactif correspond à une autre tuile. Ainsi, le moteur d'affichage met automatiquement à jour le sprite affiché dans la carte.

Propagation des effets entre leviers

Lorsqu'un levier est actionné, il peut également déclencher une modification de l'état des leviers qui lui sont liés. Cette logique est implémentée grâce à la méthode `propagate()`, qui appelle la méthode `toggle()` sur chaque levier contenu dans la liste `links`. Cependant, ce système pourrait provoquer une propagation infinie si deux leviers sont liés mutuellement. Pour éviter ce problème, la méthode `toggle()` possède un paramètre booléen appelé `initial`. Si `initial = True`, le levier est celui qui a été directement actionné par le joueur, si `initial = False`, l'action provient déjà d'une propagation. La propagation n'est déclenchée que lorsque `initial` est vrai, ce qui empêche les boucles infinies.

Intégration avec le système d'actions

L'activation d'un levier est intégrée dans le système global d'actions du jeu. Lorsqu'un joueur interagit avec un levier grâce au skill `Interact`, une `LeverAction` est créée. Cette action contient : l'identifiant du levier et l'identifiant du groupe auquel il appartient. Lorsque l'action est exécutée dans le `game_context`, elle appelle simplement la méthode `toggle()` du levier correspondant. Ce fonctionnement permet de centraliser la logique d'exécution des actions et de synchroniser les interactions en multijoueur grâce à l'envoi de deux nombres permettant d'identifier le levier. L'action est donc exécutée en quasi simultanée

Gestion de l'état final de l'énigme

La classe `Lever` contient également plusieurs méthodes utilitaires permettant de gérer l'état de l'énigme. Les méthodes `lock()` et `unlock()` permettent respectivement d'empêcher toute interaction avec le levier et de réactiver l'interaction si nécessaire.

Une fois l'énigme résolue, les leviers peuvent être verrouillés afin d'éviter toute modification ultérieure. La méthode `is_active()` permet de vérifier si un levier est activé. Elle est notamment utilisée pour déterminer si tous les leviers d'une salle sont dans l'état attendu, condition nécessaire pour résoudre l'énigme et déclencher l'ouverture des portes.

5. Implémentation de la première énigme : le système de leviers

La première énigme implémentée dans le jeu repose sur un système de leviers interactifs permettant d'ouvrir des portes lorsque certaines conditions sont remplies. Chaque levier peut être actionné par les joueurs via le système de `Skill` et d'`Action` précédemment décrit. Lorsqu'un joueur interagit avec un levier, une action est créée et exécutée, ce qui modifie son état. Un levier possède deux états : levier abaissé et levier levé. Chacune des méthodes pour générer et finir l'énigme sont contenues dans le `gameRegistry` et sont présentées à la suite.

Génération des relations entre leviers

Afin de créer une énigme intéressante, les leviers ne sont pas indépendants. La méthode spécifique `generate_random_links()` permet de créer des liens entre les leviers de manière aléatoire au sein d'un groupe. Lorsqu'un levier est actionné, il peut donc modifier l'état de lui-même et d'autres leviers liés. Pour la première difficulté de l'énigme chaque levier possède un lien aléatoire avec un autre levier et dans les niveaux plus avancés en difficultés on multiplie la quantité de levier ainsi que le nombre de lien entre eux, de manière à augmenter les possibilités et à complexifier la résolution. L'objectif est d'attendre une difficulté agréable pour les joueurs au fur et à mesure de la progression. La méthode utilisée pour générer les liens aléatoirement est la création d'une liste mélangée à partir des leviers du groupe. La méthode `shuffle` de `random` est utilisée, avec une `seed` fixe assurant le même mélange pour les deux utilisateurs. Une fois la liste créée, chacun des leviers est lié au suivant dans la liste, avec le dernier qui est lié au premier.

Mélange aléatoire des leviers

Une seconde méthode qui s'appelle `scramble_levers()` permet de mélanger l'état des leviers au lancement de la salle. Le fonctionnement est le suivant : Tous les leviers commencent dans l'état levé, correspondant à l'énigme déjà résolue. → Le système exécute ensuite une série d'actions aléatoires sur les leviers. → Chaque action modifie l'état d'un levier et potentiellement ceux qui lui sont liés. Cette méthode garantit que l'énigme est toujours résoluble, car chaque action sur un levier est réversible, il suffit de ré-interagir avec lui pour obtenir l'état précédent et l'état initial correspond à une solution valide. Ce principe est similaire au fonctionnement d'un Rubik's Cube, où le mélange consiste simplement à appliquer une suite de mouvements valides à partir de l'état résolu.

Condition de réussite de l'énigme

L'énigme est considérée comme résolue lorsque tous les leviers sont dans l'état levé. C'est `check_open_door()` qui s'occupe de gérer cette partie, à chaque action sur un levier, on utilise cette méthode pour vérifier si tous les leviers sont levés, à partir des méthodes internes à la classe `Lever`, on verrouille tous les leviers et on ouvre les portes qui ont le même ID que la salle de l'énigme. Une fois cette

condition est remplie et l'action déclenchée, les joueurs peuvent passer aux salles suivantes afin de progresser vers le bout de leur branche.

6 Détection de collisions sur des cases partielles en isométrie

Dans un moteur basé sur une grille classique, la détection de collision se fait généralement case par case : une tuile est soit traversable, soit bloquante. Cependant, dans le cas de certains objets du jeu, notamment les portes, la collision ne correspond pas à une case complète mais seulement à une portion de la tuile. Pour résoudre ce problème dans mon moteur isométrique, j'ai exploité le fait que ma fonction de conversion `iso_to_cart_tile()` renvoie initialement des coordonnées flottantes avant d'être arrondies en entiers pour accéder à la matrice de la carte. En gardant la partie flottante je peux accéder à une information particulièrement utile, la position relative à la tuile.

- la partie entière correspond à la tuile de la grille sur laquelle se trouve le joueur
- la partie décimale correspond à la position relative du joueur à l'intérieur de cette tuile

Afin d'extraire cette position relative, j'utilise un modulo 1 sur les coordonnées cartésiennes obtenues :

```
relative_x = (cart_x % 1) et relative_y = (cart_y % 1)
```

Cette opération permet de conserver uniquement la partie décimale des coordonnées, ce qui donne une position normalisée entre 0 et 1 à l'intérieur de la tuile. Grâce à ces valeurs, il devient possible de déterminer dans quelle zone de la tuile se trouve le joueur, et donc de détecter si celui-ci entre en collision avec une partie spécifique de l'objet. Par exemple, pour une porte occupant seulement une diagonale de la tuile, il est possible de définir une condition de collision dépendant de la position relative dans la case. En définissant des fonctions retournant un booléen on peut rapidement adapter les collisions à des objets partiels (n'occupant pas la surface totale de leur tuile). Par exemple pour les portes (affichées en annexe 2.1.4), je vérifie en fonction de la position de la porte si la coordonnée x ou y est supérieur ou inférieur à une valeur. La porte étant parallèle aux bords des tuiles, il suffit simplement de vérifier que le joueur se trouve sur la bonne moitié de la tuile, celle inoccupée par la porte.

Cette approche permet plusieurs avantages : conserver une grille logique simple pour la carte, gérer des collisions plus précises pour certains objets, éviter d'avoir à subdiviser les tuiles en plusieurs cases plus petites. Ainsi, le moteur reste basé sur une structure de données simple (une matrice de tuiles), tout en permettant une détection de collision plus fine adaptée aux contraintes de la perspective isométrique.

Mes futures tâches

Dans la suite du développement du projet, plusieurs axes de travail sont prévus afin d'enrichir les mécaniques de jeu et d'améliorer l'infrastructure technique. Tout d'abord, je vais développer une compétence d'attaque à distance pour le joueur, qui viendra compléter les interactions déjà existantes comme l'attaque au corps à corps ou les interactions avec l'environnement. Cette compétence devra s'intégrer dans le système de Skill et d'Action déjà mis en place afin de rester compatible avec la synchronisation multijoueur.

Je participerai également à l'implémentation de nouvelles énigmes conçues par Gustave, en l'aidant à transformer ses idées de gameplay en systèmes interactifs fonctionnels dans le moteur du jeu. Cela implique notamment la création de nouvelles interactions entre les objets de l'environnement et l'adaptation du système d'actions existant.

Enfin, je continuerai à faire évoluer le moteur isométrique afin de répondre aux nouveaux besoins du jeu. Cela pourra concerner aussi bien l'amélioration du système d'affichage, la gestion des interactions avec l'environnement ou encore l'optimisation de certaines parties du moteur pour garantir un fonctionnement fluide lors de l'ajout de nouvelles mécaniques.

Ressources

- <https://medium.com/%40aliounebfall/quest-fantasy-devlog-1-d35e71b18182>
- <https://docs.python.org/3/tutorial/classes.html>
- <https://realpython.com/python3-object-oriented-programming/>

2.3. Antoine MUH:

Durant cette période du projet, je me suis principalement occupé de la conception graphique des personnages du jeu, notamment la création de leurs sprites et de leurs animations de déplacement. Mon travail consistait à concevoir leur apparence, produire les différentes images nécessaires à leurs mouvements, puis retravailler ces images afin qu'elles puissent être utilisées correctement dans le jeu.

Cette partie du projet est importante car les sprites représentent la manière dont les personnages apparaissent à l'écran. Ils doivent donc être lisibles, cohérents entre eux et adaptés aux différentes actions que les joueurs peuvent effectuer.

1. Conception du design des personnages

Ma première tâche a été de concevoir le design des deux personnages principaux du jeu, appelés Aeden et Lyra. Ces personnages possèdent chacun des pouvoirs différents qui influencent le gameplay. Aeden est associé à la lumière, tandis que Lyra est liée à l'ombre. Lors de la création des sprites, il était donc important que leur apparence reflète cette différence, afin que les joueurs puissent facilement distinguer les deux personnages et comprendre leur rôle dans le jeu. J'ai donc cherché à créer deux designs qui restent cohérents dans le style graphique du jeu tout en ayant des caractéristiques visuelles distinctes, notamment au niveau des couleurs et de certains détails de leur tenue (visible en annexe 2.3.1 et 2.3.2)

Pour créer les premières versions des personnages, j'ai utilisé PixelLab.ai, un outil permettant de générer des images dans un style pixel art. Le pixel art consiste à créer des images à partir d'une grille de pixels visibles. Cet outil m'a permis d'obtenir une première base graphique pour les personnages, mais les images générées automatiquement ne sont pas toujours parfaitement adaptées à une utilisation directe dans un jeu vidéo. Par exemple, certains détails peuvent être peu lisibles ou certains pixels peuvent être mal placés.

Pour améliorer la qualité des sprites, je suis donc systématiquement repassé sur chaque image générée afin de la retravailler manuellement. Pour cela, j'ai utilisé Pixelorama, un logiciel spécialisé dans la création et l'édition d'images en pixel art.

Cette étape de retouche m'a permis de corriger plusieurs aspects des sprites afin d'améliorer leur qualité globale. J'ai notamment ajusté certains pixels pour améliorer la forme générale des personnages et rendre leur silhouette plus claire. J'ai également corrigé différents petits défauts visuels qui pouvaient apparaître dans les images générées. En parallèle, j'ai retravaillé certaines couleurs afin de mieux les harmoniser entre elles et d'obtenir un rendu plus cohérent. Enfin, ces modifications m'ont permis d'améliorer la lisibilité des personnages lorsqu'ils apparaissent dans le jeu, ce qui est important car les sprites sont affichés à petite taille et doivent rester facilement reconnaissables pour les joueurs.

Ce travail est important car dans un jeu vidéo, les sprites sont affichés à petite taille, et chaque pixel peut donc avoir un impact important sur le rendu final.

2. Création des animations de déplacement

Une fois les sprites statiques des personnages terminés, je me suis ensuite occupé de la création des animations de déplacement. Dans notre jeu, les personnages peuvent se déplacer dans huit directions différentes :

- Nord, Nord-est, Est, Sud-est, Sud, Sud-ouest, Ouest, Nord-ouest

Afin de représenter correctement ces déplacements, il est nécessaire de créer plusieurs images pour chaque direction. Chaque animation de marche est composée de 8 frames, c'est-à-dire plusieurs images représentant différentes étapes du mouvement. Lorsque ces images sont affichées rapidement les unes après les autres dans le jeu, cela crée l'illusion que le personnage est en train de marcher.

Pour chaque direction, j'ai donc utilisé PixelLab.ai pour générer les différentes images nécessaires à l'animation. Comme pour les sprites statiques, j'ai ensuite retravaillé chacune de ces images avec Pixelorama afin d'améliorer leur qualité et corriger certaines imperfections. La création de ces animations demande un travail assez précis. En effet, il faut veiller à ce que les différentes images soient bien alignées entre elles. Si ce n'est pas le cas, le personnage peut sembler se déplacer de manière irrégulière ou "trembler" lorsque l'animation est jouée dans le jeu. Il faut également faire attention à ce que les différentes positions du personnage restent cohérentes, notamment au niveau de la position des bras, des jambes et de la direction du corps. Cela permet d'obtenir un mouvement qui paraît plus naturel lorsque les images sont affichées successivement.

Cette étape m'a demandé de tester et d'ajuster plusieurs fois certaines images afin d'obtenir un résultat satisfaisant.

3. Organisation des sprites pour leur utilisation dans le jeu

Une fois toutes les images créées et retravaillées, j'ai également dû organiser les sprites afin qu'ils puissent être facilement utilisés dans le programme du jeu. Chaque direction de déplacement possède donc son propre ensemble d'images correspondant aux différentes étapes de l'animation. Ces images doivent être rangées de manière structurée afin que le programme puisse facilement accéder à la bonne image au bon moment. Cette organisation est importante car elle permet au code du jeu d'afficher automatiquement le bon sprite en fonction de la direction du mouvement du personnage et de son état. Pour cela on utilise des spritesheets (visible en annexe 2.3.1 et 2.3.2).

Par exemple, lorsque le joueur se déplace vers la droite, le jeu doit afficher les images correspondant à l'animation de marche vers l'est. Si le joueur s'arrête, le jeu doit afficher le sprite correspondant à la position immobile. Une bonne organisation des sprites facilite donc l'intégration des animations dans le code Python du jeu et permet d'obtenir des déplacements fluides.

Conclusion et travail restant

Durant cette période, je me suis donc concentré sur la création et l'amélioration des sprites des deux personnages, depuis leur design initial jusqu'à leurs animations de déplacement dans les différentes directions. Ce travail m'a permis de mieux comprendre les contraintes liées à la création graphique dans un jeu vidéo. J'ai également appris à utiliser des outils spécialisés dans la création et la modification de pixel art, ainsi qu'à organiser les ressources graphiques de manière structurée afin de faciliter leur intégration dans un projet de programmation.

Pour la suite du projet, il me reste encore à créer certaines animations supplémentaires, notamment celles liées aux actions des personnages comme les attaques. Ces actions nécessitent également plusieurs images afin de représenter les différentes étapes du mouvement. Je devrai également m'occuper avec Valentin de l'implémentation des musiques et des effets sonores créés par Gustave et Valentin lui-même. Cela consiste à ajouter différents fichiers audios dans le projet et à les déclencher au bon moment dans le programme, par exemple lors de certaines actions ou pour la musique d'ambiance.

Ressources

- <https://www.pixellab.ai/create-character>
- <https://www.pixellab.ai/editor>

2.4. Gustave SCHWIEN :

Depuis la dernière soutenance, j'ai travaillé principalement sur les ennemis du jeu, que l'on retrouvera surtout dans les salles de combat du sanctuaire. J'ai commencé par créer notre premier ennemi, le Slasher, à partir duquel j'ai pu développer l'algorithme de pathfinding A* comme prévu.

1. La classe Enemy

Tout d'abord, j'ai développé une classe Enemy, héritière de la classe Entity que nous avons créée pour les joueurs. Dans cette classe, j'ai regroupé toutes les variables et fonctions qui définissent un ennemi, à savoir (entre autres) :

Variables :

- Son identifiant (son index dans la liste des ennemis créés).
- Son nombre de dégâts par attaque.
- Sa portée d'attaque.
- L'état de son IA (attaquer, chasser, etc...).
- Son niveau (1, 2 ou 3) correspondant à la difficulté pour le joueur de le battre.
- Le chemin qu'il doit suivre si son IA est en mode "chasse".
- Sa direction (il y en a 8 : N-S-E-O-NE-NO-SE-SO).
-

Fonctions :

- `update_facing()` : met à jour la direction de l'ennemi en fonction de son mouvement et sa position par rapport au joueur (vecteur vitesse). La direction est déterminée en utilisant la trigonométrie avec la fonction `math.atan2(vy, vx)`, ce qui permet d'obtenir un angle (radians convertis en degrés) dont la valeur détermine la direction de l'ennemi.
- `attack()` : fonction définie dans Enemy mais réécrite dans toute classe héritière car elle dépend du type de l'ennemi.
- `chase()` : gère le mouvement de l'ennemi en fonction de sa position, celle du joueur et le chemin déterminé avec A* à partir de ces coordonnées.
- `update()` : met à jour les différentes variables de l'ennemi, pour suivre l'évolution du jeu.

La difficulté principale lors de la création de cette classe a été de différencier les variables et fonctions liées à tous les ennemis (à mettre dans Enemy) et celles qui sont spécifiques à chaque ennemi (à mettre dans les classes héritières). Il a aussi fallu modifier ces attributs à de nombreuses reprises car certains se sont révélés inutiles et d'autres indispensables durant le développement de la classe. Il est d'ailleurs encore possible qu'ils soient modifiés dans le futur ; mais grâce à la gestion des ennemis par classes héritières, les ennemis peuvent être créés de manière ordonnée et il est facile de modifier leurs propriétés donc ce n'est plus un problème.

2. La classe Slasher

À partir de la classe globale Enemy, j'ai créé le premier type d'ennemi du jeu : le Slasher. C'est un ennemi assez classique, de taille moyenne, qui se déplace à une vitesse légèrement inférieure à celle du joueur et qui lance des attaques à courte portée, similaires à des attaques de couteau ("slash"). Comme chaque ennemi possède sa propre classe, j'ai créé la classe `Slasher`, héritière de la classe `Enemy`.

Création des hitboxes du Slasher et des joueurs

Le Slasher étant un ennemi, il était nécessaire de lui associer une hitbox capable de détecter les collisions et les attaques reçues pour prendre des dégâts : c'est ce que j'ai fait à l'aide de la méthode `pygame.Rect()`, qui forme directement un rectangle correspondant à la hitbox sur l'image de l'ennemi en jeu. La hitbox se déplace avec l'ennemi en mettant à jour ses coordonnées en même temps que l'ennemi, à chaque frame. La hitbox de l'ennemi est indispensable pour que les joueurs puissent l'attaquer et l'éliminer. Sans elle, nous n'avons pas de moyen simple de détecter si une attaque a atteint un joueur.

Par ailleurs, en ajoutant une hitbox pour le Slasher, je me suis rendu compte que nous n'en avions pas encore ajouté pour les joueurs dans la classe `Player`, ce qui allait vite devenir un problème car sans hitbox les joueurs ne reçoivent pas d'attaque non plus. Par conséquent, j'en ai profité pour l'ajouter dans les attributs des objets de classe `Player`, en utilisant la même méthode que pour les ennemis.

Difficultés rencontrées avec les hitboxes

J'ai rencontré deux légers obstacles au moment de créer ces hitboxes : le premier a été qu'au départ ces dernières n'étaient pas bien centrées sur les entités. Pour m'en rendre compte, comme les `Rect` sont invisibles par défaut, j'ai dû faire des ajouts dans le fichier `game_logic.py` en créant des rectangles de couleur possédant exactement la même forme que les hitboxes avec la méthode `pygame.draw.rect`, de manière à mettre en évidence à l'écran la forme et la position des hitboxes par rapport à l'image. Ainsi, j'ai pu réaliser que les hitbox créées n'étaient pas du tout centrées sur les joueurs et ennemis, et j'ai pu corriger le problème facilement en ajoutant un offset aux coordonnées x et y des hitboxes de manière à les replacer correctement. Le deuxième obstacle que j'ai rencontré a été la forme des hitboxes. J'ai découvert ce problème en développant l'algorithme A* dont nous reparlerons plus bas. Au départ, je comptais uniquement créer des hitboxes à la forme des entités, ce qui paraît logique. Cependant, cette méthode ne peut pas fonctionner pour un jeu isométrique, car ce n'est pas un jeu en 3D mais bien en 2D. Par conséquent, la hitbox d'une entité ne devrait correspondre qu'à la forme de sa base, et non pas celle de sa hauteur. Plusieurs solutions s'offrent à nous pour résoudre ce problème : la première serait de conserver les hitboxes actuelles et d'ajouter un simple point correspondant au milieu de la base d'une entité (peu pratique), la deuxième serait simplement de reprendre la taille des hitboxes actuelles et de les adapter à la base de leurs entités, et la dernière serait de créer une nouvelle hitbox de la forme de la base pour chaque joueur et ennemi tout en conservant l'ancienne (également peu pratique car il serait lourd de créer deux hitboxes par entité). Ainsi, la solution choisie et qui sera implémentée très prochainement est la première. Ce n'est pas encore fait car nous avons résolu le problème récemment.

Développement de l'attaque du Slasher

L'attaque du Slasher se fait via la fonction `attack()`. Elle consiste à créer une zone d'attaque avec la méthode `pygame.Rect` de la longueur de sa portée au niveau de l'ennemi. Le `Rect` est créé devant l'ennemi en fonction de sa direction (pour rappel il y en a 8). Pour vérifier si l'attaque de l'ennemi touche le joueur, on utilise la méthode `collidect()` des `Rect` qui regarde si deux `Rect` (la hitbox du joueur et la zone d'attaque de l'ennemi) se touchent. Si c'est le cas, alors on appelle la fonction `take_damage()` de la classe `Entity` pour infliger les dégâts au joueur.

Pour éviter que l'attaque de l'ennemi se répète à chaque frame, on utilise une variable `attack_cooldown` incluse dans les attributs du Slasher lors de sa définition.

La fonction retourne un objet de classe `SlasherAttack` : cette classe correspond à l'action réalisée en local et sert de "relais" pour être envoyée au multijoueur. Cette partie est gérée principalement par Antoine Lapp.

3. Le pathfinding

Après avoir créé un premier ennemi, le Slasher, j'ai pu développer l'algorithme de pathfinding A*. Comme je n'en avais jamais développé auparavant, j'ai dû commencer par me familiariser avec son fonctionnement. Après avoir consulté plusieurs sites web et regardé quelques tutoriels, j'ai pu démarrer le développement dans le fichier [pathfinding.py](#).

La fonction `iswalkable()`

Cette fonction reprend le principe de la fonction `is_walkable()` de la classe `Player`, mais en plus simple. Elle prend en argument des coordonnées cartésiennes correspondant à une tuile, récupère cette tuile dans la matrice de la map, et vérifie les valeurs contenues dans la liste à l'index `[x][y]` de la tuile, correspondant au type de tuile. La fonction renvoie `True` si les valeurs ne correspondent pas à celle d'un obstacle. Mais avant de vérifier cela, on commence toujours par vérifier que les coordonnées `(x,y)` correspondent bien à une tuile dans la matrice, et ne sont pas au-delà des limites de la map.

Le `tileset` (les différents types de tuiles) étant une liste non exhaustive, il est probable qu'elle soit modifiée pour ajouter (ou supprimer) les nouveaux (ou anciens) types de tuiles. Il est facile de modifier ces types : il suffit d'ajouter les valeurs des nouvelles tuiles dans la liste des tuiles sur lesquelles on peut marcher, créée à l'intérieur de la fonction. Pour les objets posés à l'emplacement des tuiles, la méthode est similaire : il suffit d'ajouter une condition qui vérifie si la valeur dans la tuile correspond à la valeur définie pour l'objet. Cette fonction, utilisée dans l'algorithme principal, permet de différencier une tuile que l'ennemi peut traverser d'un obstacle, afin de calculer le meilleur chemin possible.

La fonction `heuristic()`

Cette fonction renvoie la distance de Tchebychev entre deux points `a` et `b` de coordonnées `(x1,y1)` et `(x2,y2)`. C'est une fonction indispensable pour estimer le coût théorique d'un parcours de `a` à `b`, qui fait partie de la logique fondamentale du A* : on se trouve sur une tuile et on connaît le coût du trajet déjà parcouru depuis le départ jusqu'ici, appelé `g`. La fonction `heuristic()` permet de calculer le coût estimé du parcours en partant d'une tuile voisine, appelé `h`. Le coût total du trajet est alors $f = g + h$. Pour chaque tuile, on estime ce coût total, et on choisit la tuile dont le coût est le plus faible. Cela explique l'importance de la fonction `heuristic()`.

La classe `Node`

Pour clarifier l'algorithme, on représente une tuile par un nœud, un objet de classe `Node`. Un objet de ce type possède :

- Une variable `position` contenant les coordonnées `(x,y)` de la tuile correspondant à ce nœud.
- Une variable `cost` contenant le coût du chemin parcouru jusqu'ici (le `g`).
- Une variable `heuristic` contenant le coût estimé depuis ce nœud jusqu'à la cible (le `h`).
- Une variable `parent` contenant le nœud d'où vient le nœud actuel (pour récupérer le chemin final lorsque la cible est atteinte).
- La fonction `__lt__()` qui vérifie si la somme du $g + h (= f)$ d'un nœud est inférieure à celle d'un autre nœud, afin de trouver le meilleur parmi les voisins du nœud actuel.

Un nœud correspond donc à une tuile, et regroupe toutes les informations nécessaires au calcul du chemin.

L'algorithme principal : `astar()`

C'est la fonction principale qui calcule et renvoie le chemin sous forme de liste de coordonnées de tuiles. Il part de deux listes, l'une contenant les tuiles déjà parcourues (`closed_set`), l'autre contenant les différentes tuiles à explorer (`open_set`), et un dictionnaire appelé `g`, qui contient les plus petits coûts totaux `f` pour chaque nœud en partant du nœud de départ.

Tant qu'il y a des éléments dans `open_set`, on récupère son premier élément, correspondant au nœud avec la plus petite somme $cost + heuristic$ grâce à la librairie `heapq`. On vérifie ensuite si les coordonnées du nœud correspondent à celles de la cible (ce qui voudrait dire que le chemin a atteint son objectif). Si c'est le cas, alors on crée une liste vide `path`, on ajoute les coordonnées du nœud actuel, et on met à jour ce nœud avec son parent se trouvant dans ses attributs : en répétant ce processus jusqu'à atteindre `None` (premier nœud), on récupère le chemin parcouru et on le met dans la liste `path`. Si le nœud n'est pas la cible, on prend un par un les nœuds correspondant aux tuiles voisines de la tuile actuelle, et on vérifie qu'elles sont traversables avec `iswalkable()`. Si c'est une tuile diagonale, on vérifie aussi les tuiles adjacentes, car si elles ne sont pas aussi traversables alors l'ennemi ne peut en aucun cas passer par là. Enfin, on vérifie si ce sont des tuiles qui ont déjà été explorées, pour ne pas passer deux fois au même endroit et ainsi ne pas créer de boucle infinie. On associe également un coût légèrement plus élevé à un déplacement en diagonale qu'à un déplacement vertical ou horizontal pour rendre le chemin plus logique. Après ces vérifications passées, on regarde si le coût total f pour atteindre le nœud actuel est plus petit que le f retenu pour ce même nœud. Si c'est le cas, on le remplace dans le dictionnaire `g`, on crée un nouveau nœud enfant en prenant le nœud actuel comme nœud parent, et on ajoute ce nouveau nœud à l'`open_set` avec la méthode `heapq.heappush()`. Si la liste `open_set` se retrouve vide et qu'aucun chemin n'a été trouvé, on renvoie `None` donc l'ennemi ne se déplace pas.

Pour éviter les crashes et les longs calculs inutiles, nous avons ajouté une limite d'itérations dans la boucle principale qui explore chaque nœud, partant du principe qu'au-delà de cette limite si aucun chemin n'a été trouvé alors on considère que le nombre de calculs est trop élevé pour calculer un chemin sans trop prendre sur le processeur. Sans cela, un ennemi pourrait théoriquement calculer un chemin d'un coin à l'autre de la map, ce qui ferait beaucoup ramer le jeu. La limite par défaut a été mise à 800 pour l'instant, mais elle pourra être adaptée au moment de la création des salles de combat.

Cet algorithme nous permet de trouver un chemin optimisé, en évitant les obstacles pour l'ennemi et les calculs trop lourds pour le processeur. Ainsi, l'ennemi peut désormais bouger et se déplacer en autonomie vers le joueur.

Une autre fonction utile pour la suite : `bresenham()`

J'ai développé une fonction `bresenham()` qui prend en argument deux points `a` et `b` de coordonnées isométriques $(x1, y1)$ et $(x2, y2)$, et qui renvoie la liste de coordonnées cartésiennes de chaque tuile traversée par la ligne entre les points `a` et `b`.

Cette fonction sera particulièrement utile pour les ennemis qui attaquent de loin en tirant des projectiles comme le Flinger, le prochain ennemi à développer : elle permettra de vérifier s'il existe une trajectoire pour le projectile de l'ennemi partant de ce dernier et atteignant le joueur sans toucher d'obstacle, donc si l'ennemi peut tirer sur le joueur.

Conclusion et tâches futures

Depuis la dernière soutenance, j'ai pu démarrer pleinement le développement des parties du jeu dont je suis responsable. Nous pouvons créer facilement de nouveaux ennemis, et le plus gros du travail sur leur IA a été réalisé. Il reste maintenant à peaufiner les fonctions pour corriger les légers bugs de logique et visuels, et à ajouter quelques détails. Maintenant, je vais pouvoir ajouter de nouveaux ennemis, notamment le Flinger, qui attaquera le joueur en lui tirant des projectiles à distance, et un autre qui infligera des dégâts de zone tout autour de lui. Je vais également me pencher plus sérieusement sur le développement des énigmes et des salles de combat avec Guillaume. En parallèle, je vais travailler sur la soundtrack du jeu, sur laquelle j'aide Valentin.

Ressources

- Documentation `pygame.Rect()` : <https://www.pygame.org/docs/ref/rect.html>
- Documentation module `math` : <https://docs.python.org/3/library/math.html>
- Tutoriel A* en python de DrCodie : <https://youtu.be/qWbqHKzVLlw?si=FfGjaxWZRLBU6wpz>
- Exemple d'implémentation du A* en Python : <https://www.alps.academy/a-star-algorithm-python/>
- Autre ressource sur l'algorithme A* (Wikipedia) : https://en.wikipedia.org/wiki/A*_search_algorithm

2.5. Valentin GOUSSOT :

Pour cette deuxième soutenance, je me suis occupé du design des tiles pour les décors et j'ai également avancé sur les musiques de combat et d'énigmes qui seront implantés pour la dernière soutenance.

1. Design des tiles

Tout d'abord, j'ai commencé par chercher des designs adéquats pour les tiles du jeu. Les tiles sont de petites images, généralement carrées, qui servent de base pour construire les décors dans un jeu vidéo. Elles représentent par exemple le sol, les murs, l'herbe, les chemins, l'eau ou encore différents éléments du paysage. Au lieu de dessiner un décor entier d'un seul coup, j'utilise ces images répétables que j'assemble comme des pièces de puzzle. Cette technique permet de créer des environnements complets tout en gardant une bonne organisation.

Dans ce projet, chaque tile a une taille de 64 × 64 pixels. Cela signifie que chaque image est composée d'une grille de 64 pixels de largeur et 64 pixels de hauteur. Utiliser une taille fixe permet au moteur de jeu de placer les tiles correctement sur la carte et de créer des décors alignés et cohérents. Ce format est aussi suffisamment grand pour ajouter des détails tout en restant adapté à un style graphique en pixel art.

Pour créer le design de ces tiles, j'ai utilisé plusieurs logiciels, notamment LibreSprite, Piskel et Pixellab avec certaines fonctionnalités d'intelligence artificielle. Ces outils me permettent de dessiner en pixel art, un style graphique où les images sont créées pixel par pixel. Avec LibreSprite et Piskel, j'ai pu dessiner précisément les différents éléments des décors et vérifier qu'ils s'assemblent correctement. Pixellab m'a également aidé pour certaines parties du design et pour tester différentes idées visuelles.

Lors de la création des tiles, je dois faire attention à plusieurs éléments : les couleurs utilisées, la cohérence du style graphique et surtout les bords des tiles. Lorsque deux tiles sont placées côte à côte dans le jeu, elles doivent se connecter naturellement pour éviter que le joueur remarque des coupures ou des décalages dans le décor. Cela demande donc de la précision et plusieurs tests pour vérifier que les tiles fonctionnent bien ensemble une fois intégrées dans le jeu. Je dois également créer plusieurs tiles différentes pour une seule partie du décor pour éviter un effet répétitif dans le visuel du jeu.

2. Musiques de combats et d'énigmes

En parallèle, j'ai également continué à travailler sur la musique du jeu. J'ai continué la composition des musiques de combat, qui doivent être dynamiques et rythmées afin de transmettre un sentiment d'action et d'intensité lorsque le joueur affronte des ennemis. L'objectif est de créer une musique qui soutient le rythme du combat et qui rend ces moments plus immersifs.

Enfin, j'ai composé plusieurs musiques destinées aux énigmes. Contrairement aux musiques de combat, celles-ci doivent créer une atmosphère plus calme et réfléchie. Elles sont conçues pour accompagner le joueur pendant qu'il cherche une solution, sans être trop rapides ou trop stressantes. J'ai donc utilisé des mélodies plus douces et une ambiance légèrement mystérieuse pour encourager la réflexion tout en restant agréable à écouter.

3. Conclusion et tâches restantes

Ce travail sur les graphismes et la musique me permet donc de participer activement à plusieurs aspects du développement du jeu. Il contribue à construire à la fois l'environnement visuel du jeu grâce aux tiles et l'ambiance sonore grâce aux différentes musiques créées pour accompagner les combats et les moments de réflexion du joueur.

Pour la suite du projet, il me reste encore plusieurs éléments à terminer pour améliorer le jeu. Je dois finaliser les musiques pour les énigmes et continuer à développer les musiques de combat pour qu'elles correspondent mieux aux différentes situations du jeu.

Je dois également créer davantage de design pour les tiles. L'objectif est d'ajouter plus de variantes dans les décors afin d'éviter que l'environnement du jeu paraisse trop répétitif pour le joueur.

Enfin, il me reste à implémenter le sound design avec Antoine MUH dans le jeu, c'est-à-dire intégrer les effets sonores directement dans le moteur de jeu pour qu'ils se déclenchent au bon moment (par exemple lors des déplacements, des actions ou des interactions). Cette étape est importante pour rendre l'expérience de jeu plus immersive et complète.

Ressources

- <https://www.pixellab.ai/create>
- <https://www.piskelapp.com/p/create/sprite/>
- <https://libresprite.github.io/#!/>

3. Récit de mise en œuvre du projet

3.1. Challenges

La complexité de la synchronisation multijoueur

L'un des principaux défis rencontrés durant le développement concerne la synchronisation du jeu en multijoueur coopératif. Contrairement à un jeu solo, chaque action réalisée par un joueur doit être répercutée de manière cohérente sur l'ordinateur de l'autre joueur afin de maintenir un état du monde identique pour les deux en personnes en coopération, le tout avec le moins de latence possible.

Ce problème devient particulièrement complexe avec l'environnement qui devient de plus en plus complet, les interactions se multiplient, avec les énigmes et les ennemis. La mise en place d'un système basé sur des actions sérialisées et envoyées sur le réseau a permis de répondre à ce problème, mais cela a nécessité une importante quantité de tests et de débogage ainsi que de nombreuses réflexions et discussions au sein du groupe.

Gestion collaborative du code avec Git

Le travail en équipe a également présenté certains défis techniques et organisationnels. L'utilisation de Git pour gérer les différentes versions du projet nous a parfois confrontés à des conflits de fusion (merge conflicts) lorsque plusieurs membres modifiaient les mêmes fichiers. Ces situations ont amené une première expérience de collaboration sur un projet de développement partagé, ce qui nous a obligé à améliorer notre manière de

travailler, notamment en communiquant davantage avant de modifier certaines parties du code effectuant des commits plus réguliers utilisant des branches pour isoler certaines fonctionnalités. Nous avons dû nous entraider afin que ceux qui avaient le plus d'expérience avec git apprennent aux autres à s'en servir. Même si ces conflits ont parfois ralenti le développement, ils nous ont permis de mieux comprendre les bonnes pratiques de développement collaboratif.

Organisation du travail en parallèle des études

Un autre défi important a été la gestion du temps. Le développement du projet devait être mené en parallèle des autres cours, des travaux pratiques et des périodes d'examens. Cela a parfois limité le temps que nous pouvions consacrer au projet et nous a obligés à prioriser certaines fonctionnalités plutôt que d'essayer d'implémenter toutes les idées initialement prévues. Cette contrainte nous a amenés à mieux organiser le travail et à définir des objectifs plus réalistes pour chaque phase du projet.

3.2. Réussites / succès

Coopération efficace au sein de l'équipe

Malgré les difficultés rencontrées, le projet a été marqué par une collaboration productive entre les membres de l'équipe. La répartition des tâches a permis à chacun de travailler sur des aspects spécifiques du jeu, comme le moteur isométrique, les mécaniques d'énigmes ou les systèmes d'interaction. Cette organisation nous a permis d'avancer efficacement tout en partageant régulièrement nos progrès et nos difficultés. Cette expérience nous a également permis de développer nos compétences en travail d'équipe sur un projet informatique. Cette coopération a été rendue possible grâce à des réunions en appel afin de faire le point sur les tâches à réaliser, mais aussi grâce à la liste de tâche notion et à la communication hebdomadaire sur notre serveur discord.

Adoption des bonnes pratiques Git

Par le biais de nos nombreuses problématiques de développement, nous avons progressivement découvert et adopté les pratiques d'utilisation de Git afin de faciliter le travail en groupe. Au début du projet, certaines modifications étaient envoyées directement sur la branche principale, ce qui compliquait le suivi des changements et augmenter le risque de bug bloquant. Nous utilisons maintenant des branches pour structurer les modifications et le développement des nouvelles fonctionnalités. Une fois les modifications validées par plusieurs personnes du groupe nous fusionnons les branches secondaires vers la branche principale. Ceci nous permet d'avoir une branche principale toujours fonctionnelle et sans bugs bloquants. Elle est donc toujours disponible pour une éventuelle démonstration.

Pas de bug bloquant

L'une des principales réussites du projet est d'avoir réussi à développer jusqu'à là un prototype jouable du jeu, intégrant la première énigme et les prémices des ennemis ainsi que de la carte. Le moteur isométrique est fonctionnel, le système d'action et compétences nous permettent de développer la suite des interactions. C'est une étape importante de faite.

4. Annexes

1.1. Tableau extrait du TSD avec estimations de l'avancée aux différentes soutenances

Task progress table			
Tâches	Soutenance 1	Soutenance 2	Sout. Finale
Pixel art	20%	50%	100%
Musique	0%	50%	100%
Animation	0%	50%	100%
sound design	0%	50%	100%
multijoueur	100%	100%	100%
gameplay	50%	90-100%	100%
map	20%	50%	100%
déroulement du jeu	80%	100%	100%
affichage jeu	80%	100%	100%
developement des joueurs	70%	90%	100%
affichage menu	30%	100%	100%
AI	20-30%	70%	100%

Veillez indiquer le pourcentage d'avancement prévu à la date de la soutenance



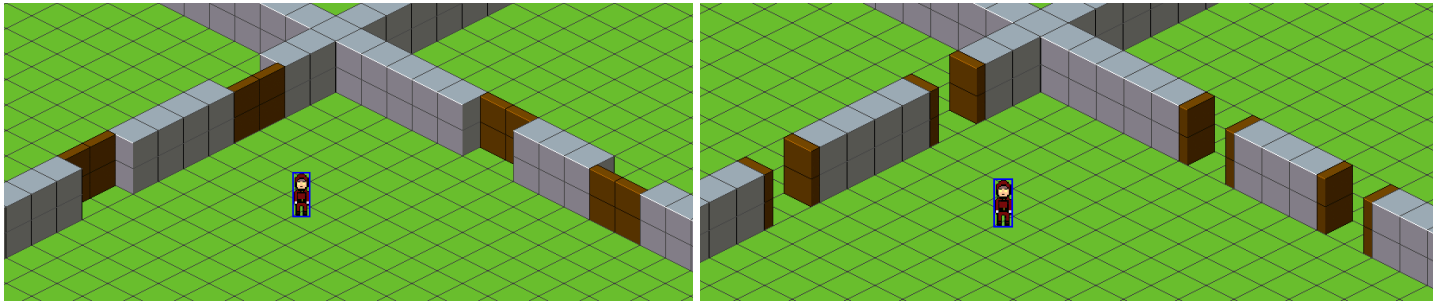
2.1.1. Le menu pause



2.1.2. La barre de vie

	Nord-Est	Sud-Ouest	Nord-Ouest	Sud-Est
Fermée	20	21	22	23
Ouverte	24	25	26	27

2.1.3.



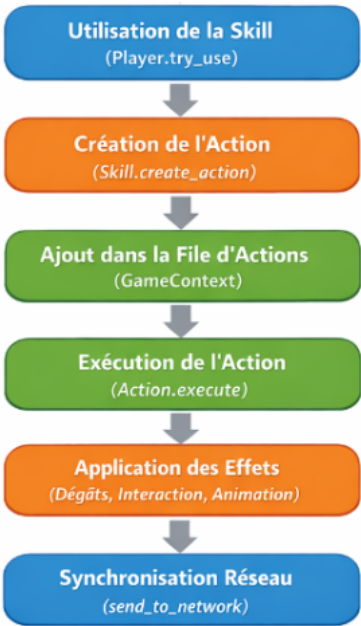
2.1.4. Les portes ouvertes et fermées



2.3.1. Personnage : Aeden



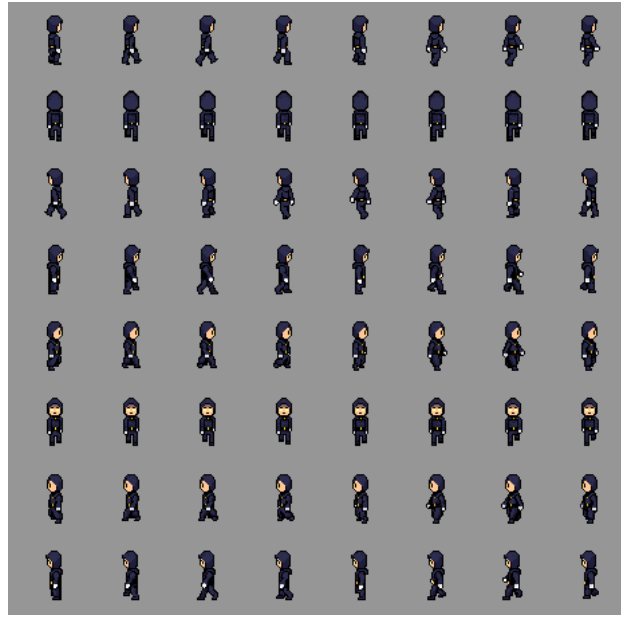
Spritesheet du personnage en mouvement



2.2.2. Schéma de l'architecture des actions



2.3.2 Personnage : Lyra



Spritesheet du personnage en mouvement